

Plan de Gestión de Configuración de Software (SCM)

Proyecto: Game-Hub & Services Ecosystem

Responsable: Fernando (Gestor de Proyecto)

1. Herramientas y Repositorio

- Herramienta de Control de Versiones: Se utilizará **GitHub** como plataforma centralizada para el control de versiones, permitiendo el trabajo colaborativo y el seguimiento de cambios mediante Git.
- URL del Repositorio: <https://github.com/Fernando01goumh/Practica-GameHub-Grupo-4>.
- Gestión de Tareas: Para la sincronización de las tareas definidas en la WBS, se empleará GitHub Projects. Esta herramienta permitirá vincular directamente las *Issues* con el código y realizar un seguimiento visual del progreso de cada integrante del equipo.

2. Política de Ramas (Branching Strategy)

Para garantizar la estabilidad del código y evitar conflictos, el proyecto seguirá el modelo GitFlow. Se establecen las siguientes ramas obligatorias:

- main: Contiene únicamente código estable, probado y listo para la entrega final de cada fase.
- develop: Rama principal de integración. Aquí se fusionarán todas las nuevas funcionalidades antes de pasar a la rama principal.
- feature/nombre-tarea: Ramas temporales creadas para el desarrollo de funcionalidades específicas de la WBS (ej. feature/CU-01-registro). Al finalizar la tarea, se realizará un *Pull Request* hacia develop.
- fix/nombre-error: Ramas destinadas exclusivamente a la corrección de fallos críticos detectados en develop.

3. Política de Commits

Como gestor, se exige disciplina en el historial del repositorio para asegurar la trazabilidad del proyecto:

- Frecuencia: Los commits deben realizarse de forma regular tras completar unidades de trabajo lógicas. Se prohíbe subir bloques masivos de código al finalizar la fase.
- Mensajes Descriptivos: Los mensajes deben ser claros y explicar brevemente el cambio realizado (ej. "feat: implementada validación de edad en registro").

- Vinculación con la WBS: Cada commit debe incluir obligatoriamente el ID de la tarea de la WBS que resuelve para facilitar la auditoría de trabajo (ej. feat(2.2.1): implementación CU-01 #WBS-2.2.1).

4. Copias de Seguridad y Recuperación

Para mitigar el riesgo de pérdida de información, se establece el siguiente protocolo:

- Procedimiento de Backup:
 1. Redundancia en la Nube: GitHub proporciona la copia principal distribuida.
 2. Clones Locales: Cada miembro del equipo mantendrá una copia actualizada en su equipo local al realizar *pulls* diarios.
 3. Exportación Semanal: El gestor realizará una exportación semanal del repositorio en formato comprimido (.zip) que se almacenará en un servicio de almacenamiento en la nube secundario (Google Drive/OneDrive).
- Plan de Acción ante Fallos: En caso de corrupción del repositorio central o pérdida de datos, se restaurará la versión más reciente a partir de la última exportación semanal o, en su defecto, desde el clon local más actualizado de los desarrolladores.